

Non-Primitive Data Type or Reference Data Types	2
Single-Dimensional Arrays	3
Demonstrate Array	5
Strings	6
Demonstrate Strings	9
Objects Oriented Programming	11
Class and Object	13
Example Blueprint	14
Instantiating: Making Objects from Your Class	14
What Happens in Computer Memory During Instantiation?	14
Encapsulation	15
Here is a complete, working example	16
Run the code and see how the modifiers react.....	18
Getter and Setter method.....	20
This keyword.....	22
Visualizing this in Memory	23
Is this mandatory in Getters?.....	23
Inheritance.....	24
Abstract Class.....	26
Polymorphism	28
Polymorphism in Action (Code Example)	28
Variable scope	30
Project - Build a Student Grade & Honor Roll Tracker.....	32
Task 1: Read messy Student Names from user	32
Task 2: Request user to input Math and Science grade and calculate the average	34
Task 3: Scale Up to a Full Schedule.....	35
Task 4: Clean the Main Architecture (Introducing Multiple Functions).....	36
Task 5: OOPs concepts – class, encapsulation	38
Task 6: OOPs concepts – Data hiding.....	40
Task 7: OOPs – Inheritance and overriding	43
Task 8: OOPS – Polymorphism	46
Task 9: File Input Output	49
Task 10: Read back from on start-up.	52

Non-Primitive Data Type or Reference Data Types

The Reference Data Types will contain a memory address of variable values because the reference types won't store the variable value directly in memory. They are strings, objects, arrays, etc.

Primitive data type vs. non-primitive type in Java

Primitive Datatypes	Non-Primitive Datatypes
Primitive Datatypes are pre-defined by the language itself.	Non-Primitive Datatypes need to be defined by the user.
When declared primitive data is stored in the stack.	The reference variables are stored in the stack but the original object is in the heap.
When a copy is created for the data, a completely separate allocation is done for the copy.	Copies here create new reference variables but they both point to the same object in the heap.
Thus, the changes made to the copy are not reflected in the original.	Here however the changes made to the copy trace back to the original.
The primitive data structure will contain some value, i.e., it cannot be NULL.	The non-primitive data structure can consist of a NULL value.
The size depends on the type of data structure.	In the case of a non-primitive data structure, size is not fixed.
The primitive data structure can be used to call the methods.	Non-primitive data structure cannot be used to call the methods.

Single-Dimensional Arrays

Lesson link: https://www.w3schools.com/java/java_arrays.asp

- An **array** is an ordered, fixed-size container that allows you to store multiple values of the **same data type** inside a single variable name.
- Instead of creating ten separate variables to hold ten student test scores (score1, score2, etc.), you create one single array variable named scores that holds all ten values together in a neat list.

1. Visualizing Arrays and Indexes

Like strings, arrays use a zero-based numbering system to track where items live. Each slot in the array is called an **element**, and its coordinate position is called its **index**.

Let's look at an integer array containing four numbers: `[10, 20, 30, 40]`

text

Element Value:	10	20	30	40
Index Number:	0	1	2	3

Use code with caution.

- **Array Length:** 4
- **First Element:** Index 0 (10)
- **Last Element:** Index 3 (40) — *Always the Length minus 1!*

2. Creating Arrays in Java

There are two distinct ways to declare and initialize an array in Java:

Method A: The Literal Way (When you already know the values)

Use curly braces `{}` to plug data straight in when creating the variable.

java

```
int[] luckyNumbers = {7, 13, 21, 42};
String[] fruits = {"Apple", "Banana", "Orange"};
```

Use code with caution.

Method B: The Blank Blueprint Way (When you don't know the values yet)

Use the `new` keyword and declare an absolute, fixed size. Java will build an empty list filled with default values (like zeros). **You cannot resize it later.**

java

```
// Builds an empty list with room for exactly 5 integers
int[] studentGrades = new int[5]; // Stores: [0, 0, 0, 0, 0]
```

3. Reading and Writing Array Values

To look inside a slot or change a value, use square brackets `[]` paired with the target index number. 

java

```
int[] scores = {90, 85, 77};

// 1. READ a value from index 1
System.out.println(scores[1]); // Prints: 85

// 2. WRITE/UPDATE a value at index 2
scores[2] = 95; // Changes 77 to 95

// Array is now: [90, 85, 95]
```

4. Processing Arrays with Loops

Because arrays rely on sequential numbers (0, 1, 2, 3...), they are perfect partners for **for loops**. The loop counter acts as the index pointer to scan through the elements automatically. 

Standard For Loop (Best for modifying or tracking indexes)

Every array has a built-in `.length` attribute that tells you how many items are inside. 

java

```
int[] runTimes = {12, 15, 11, 14};

for (int i = 0; i < runTimes.length; i++) {
    System.out.println("Runner " + i + " finished in " + runTimes[i] + " mins");
}
```

Enhanced For Loop / "For-Each" (Best for quickly reading data)

If you only need to look at every value without tracking index coordinates, use this clean syntax. It reads aloud as: "For every individual `time` inside the `runTimes` list..."

java

```
int[] runTimes = {12, 15, 11, 14};

for (int time : runTimes) {
    System.out.println("Time: " + time);
}
```

Demonstrate Array

Example demonstrating how to **declare, initialize, and loop through** an array in Java.

```
public class ArrayDemo {
    public static void main(String[] args) {
        // 1. Declare and initialize an array of Strings with a size of 4
        String[] fruits = {"Apple", "Banana", "Cherry", "Mango"};

        // 2. Access elements using an index (Note: Java arrays start at index 0)
        System.out.println("First fruit: " + fruits[0]); // Outputs: Apple
        System.out.println("Third fruit: " + fruits[2]); // Outputs: Cherry

        // 3. Modify an existing element
        fruits[1] = "Blueberry"; // Changes "Banana" to "Blueberry"

        // 4. Get the size of the array
        System.out.println("\nTotal items in array: " + fruits.length);

        // 5. Loop through the array using an enhanced 'for-each' loop
        System.out.println("--- Current Fruit List ---");
        for (String fruit : fruits) {
            System.out.println("- " + fruit);
        }
    }
}
```

Expected Output

First fruit: Apple
Third fruit: Cherry

Total items in array: 4
--- Current Fruit List ---
- Apple
- Blueberry
- Cherry

Strings

Lesson link

https://www.w3schools.com/java/java_strings.asp

- Strings are defined as an array of characters / sequence of characters that is held in single variable. Used to represent text (like names, passwords, or sentences).
- Character array (not String) is a collection of separate char-type entities accessed by indexing.
- Unlike C/C++, Java strings are not terminated with null characters.
- **Syntax:** Declaring a string

- `<String_Type> <string_variable> = "<sequence_of_string>";`
- `String s1 = "Hello";` // Stored in String Pool
- `String s2 = "Hello";` // Reuses the same object from the String Pool
- `String s3 = new String("Hello");` // Creates a new String object outside the Pool

- The Key Rule: Strings are Immutable

That is once String is created, **it can never be changed in memory.**

String manipulation methods that "modify" text, Java doesn't alter the original word. Instead, it quietly creates a brand-new String in the background and hands you fresh copy.

- Strings is object and provides built-in function (methods) to search, cut, clean, and modify text.
- Built-in String operation functions are, such as finding length, extracting substrings, replacing characters, and more.

Operation	Method
Get Length	<code>length()</code>
Get Character	<code>charAt(int index)</code>
Extract Substring	<code>substring(int start, int end)</code>
Find Substring	<code>indexOf(String substr)</code>
Replace Characters	<code>replace(char old, char new)</code>
Change Case	<code>toUpperCase(), toLowerCase()</code>
Split String	<code>split(String regex)</code>

- Built-in String comparing functions

Golden Rule of Comparison, use `equals()` Instead of `==`. The `==` operator compares references, not the actual content of the strings, whereas `equals()` compares the values.

Method	Purpose
<code>equals()</code>	Compares content
<code>equalsIgnoreCase()</code>	Ignores case differences
<code>compareTo()</code>	Lexicographical comparison

- Substring functions

Extracts a smaller piece of text out of a larger String. It takes a starting index and an optional ending index. *Note:* The ending index is **exclusive** (Java stops right *before* that number).

2. Visualizing String Indexes

To manipulate text, you must understand how Java views a String. Java assigns a number position (called an **index**) to every single character, **starting exactly at 0**. Spaces count as characters too! 

Let's look at the index mapping for the text "Java App" :

text										
Character:		J		a		v		a		
										A
										p
										p
Index:		0		1		2		3		4
										5
										6
										7

Use code with caution. 

- **Total Characters (Length):** 8
- **First Character Index:** 0 ('J')
- **Last Character Index:** 7 ('p') — *Always Length minus 1!* 

```
String phrase = "Superstar";

// Cut from index 5 all the way to the end String
endPiece = phrase.substring(5); // "star"

// Cut from index 0 up to (but not including) index 5
String frontPiece = phrase.substring(0, 5); // "Super"
```

- Advance methods

```
String.format() allows you to create formatted strings dynamically.
String formatted = String.format("Name: %s, Age: %d", "Alice", 30);
System.out.println(formatted); // "Name: Alice, Age: 30"

String joined = String.join("-", "2023", "12", "31"); // "2023-12-31"

String s = " Hello ";
System.out.println(s.strip()); // "Hello" Removes leading and trailing whitespace.
System.out.println(" ".isBlank()); // true
System.out.println("Hi".repeat(3)); // "HiHiHi"

Regular Expressions & Pattern Matching
String email = "user@example.com";
boolean isValid = email.matches("^[\\w.-]+@[\\w.-]+\\.\\[a-zA-Z]{2,6}$");
```

- Built-in Java methods used to modify

```
String CSVData = "Pizza,Burgers,Tacos";
String[] foodList = CSVData.split(",");

// foodList[0] is now "Pizza"
// foodList[1] is now "Burgers"
// foodList[2] is now "Tacos"
```

Method Signature	What It Does (Action)	Input Example	Output Result
<code>.trim()</code>	Removes all blank spaces from the very beginning and very end.	<code>" Hello World ".trim()</code>	<code>"Hello World"</code>
<code>.toLowerCase()</code>	Converts every letter to lowercase.	<code>"Java123".toLowerCase()</code>	<code>"java123"</code>
<code>.toUpperCase()</code>	Converts every letter to uppercase.	<code>"code".toUpperCase()</code>	<code>"CODE"</code>
<code>.replace(old, new)</code>	Swaps out a target character or word with a new one.	<code>"apple".replace('p', 'b')</code>	<code>"abble"</code>
<code>.substring(start)</code>	Cuts text starting from the chosen index to the very end.	<code>"Superstar".substring(5)</code>	<code>"star"</code>
<code>.substring(start, end)</code>	Cuts text from the <code>start</code> index up to but not including the <code>end</code> index.	<code>"Superstar".substring(0, 5)</code>	<code>"Super"</code>
<code>.concat(str)</code>	Glues another string to the end of the current string.	<code>"Ice".concat("Cream")</code>	<code>"IceCream"</code>

Demonstrate Strings

Example demonstrating how to use **Java Strings**, showcasing common operations like finding length, changing case, checking contents, and joining text.

```
public class StringDemo {
    public static void main(String[] args) {
        // 1. Declare and initialize a String
        String text = " Hello, Java Learners! ";

        // 2. Get the length of the string
        System.out.println("Original length: " + text.length());

        // 3. Remove leading and trailing whitespace using trim()
        String cleanText = text.trim();
        System.out.println("Cleaned text: '" + cleanText + "'");

        // 4. Convert case (Uppercase / Lowercase)
        System.out.println("Uppercase: " + cleanText.toUpperCase());

        // 5. Extract a portion of the text using substring(startIndex, endIndex)
        // Extracts characters from index 7 up to (but not including) index 11
        String word = cleanText.substring(7, 11);
        System.out.println("Extracted word: " + word); // Outputs: Java

        // 6. Check if the string contains a specific word
        boolean hasJava = cleanText.contains("Java");
        System.out.println("Contains 'Java'? : " + hasJava);

        // 7. Replace characters or words
        String newText = cleanText.replace("Learners", "Experts");
        System.out.println("Replaced text: " + newText);

        String s1 = new String("Apple");
        String s2 = new String("Apple");

        System.out.println(s1 == s2); // ✗ Prints false (different memory objects)
        System.out.println(s1.equals(s2)); // Prints true (same characters)
    }
}
```

Expected Output

Original length: 25
Cleaned text: 'Hello, Java Learners!'
Uppercase: HELLO, JAVA LEARNERS!
Extracted word: Java
Contains 'Java'? : true
Replaced text: Hello, Java Experts!

Crucial Tip for Beginners: Comparing Strings

In Java, **never use ==** to compare the text content of two Strings. Always use the **.equals()** method.

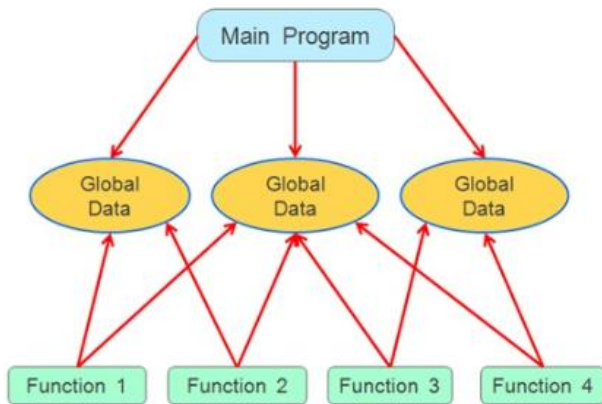
- == checks if the strings are the exact same **object in memory**.
- .equals() checks if the **characters inside** the strings match.

Objects Oriented Programming

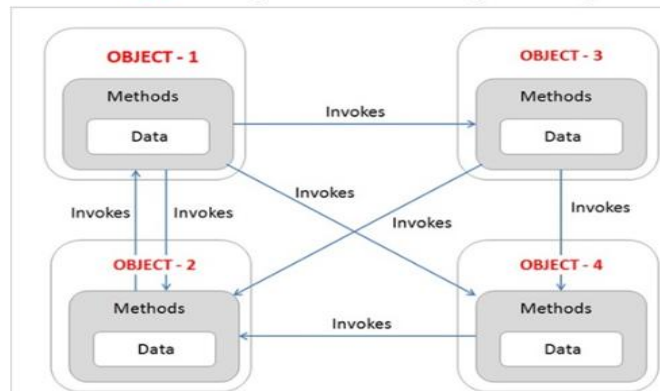
OOP, we can create modular, reusable, and maintainable code, making programming more efficient.

- In Procedural programming, it is difficult to secure data. Data is global and all functions have access to data.
- Fundamental concept of OOP is Class and Object. Classes serve as blueprints for creating objects.
- Each idea is modeled s class. Objects of classes interact to solve the problem.
- Other class does not have access to data of another ideas class. Data is protected and only member functions have access to data.

Procedural programming



OOP - Object Oriented Programming



Principles Of Object Oriented Programming

The OOP programming organization is based on the four major fundamental principles . The OOP principles include :

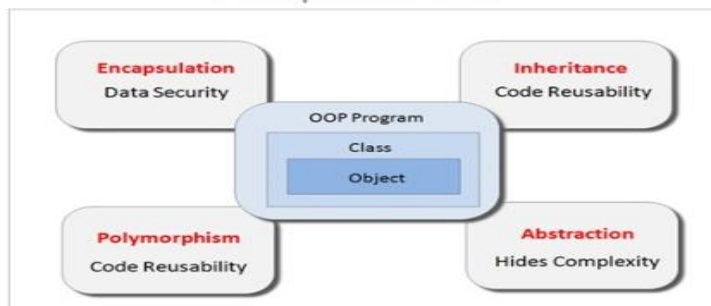
1. Encapsulation

2. Inheritance

3. Abstraction

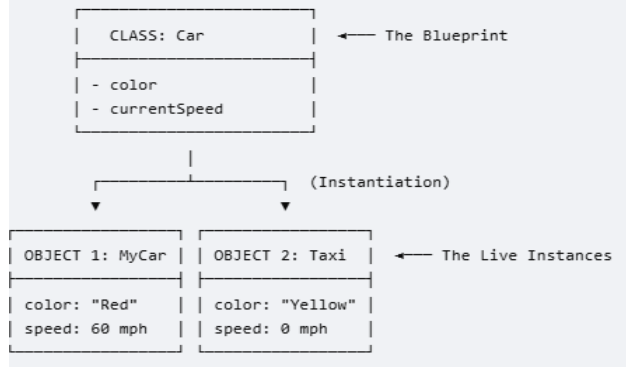
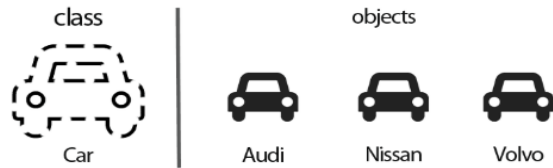
4. Polymorphism

Principles Of OOP



1> Abstraction

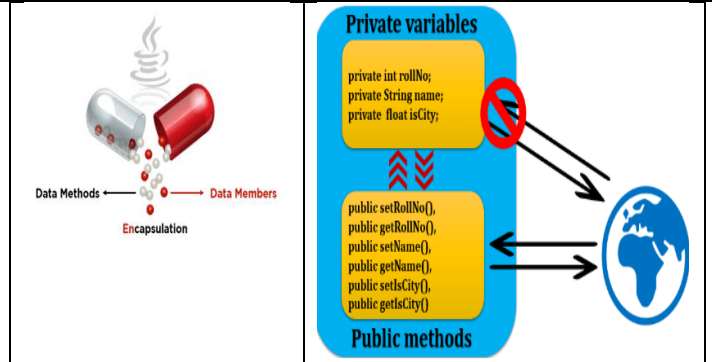
Abstraction: Simplifying Complexity by providing blueprint for expected behaviors and attributes.

**Classes: Blueprints for Objects****2. Encapsulation: Bundling Data and Logic**

*Powerful concept that combines data and logic into cohesive units.

* Data hiding: Encapsulation safeguards the inner workings of objects, ensuring that they are accessible only through defined interfaces while hiding the intricate complexities.

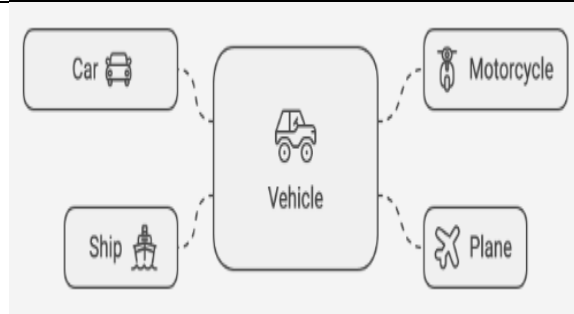
This is done by providing access modifier like Public, Private, Protected to data and methods

**3. Inheritance**

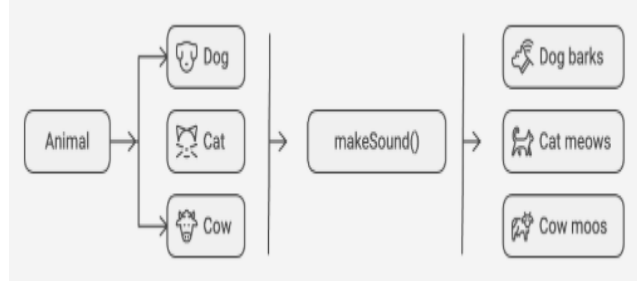
*Inheritance allows you to create new classes based on existing ones.

* This helps **reduce code duplication** (reuse) and makes extending the code easier.

*The new class (child) inherits all properties and methods from the **parent class** but can also add its own.

**4. Polymorphism**

Polymorphism allows objects of different classes to be treated as objects of a **common superclass**. It enables methods to do **different things** based on the object they are acting upon. It's like a **universal remote control** where one button can perform different functions depending on which device you're using.

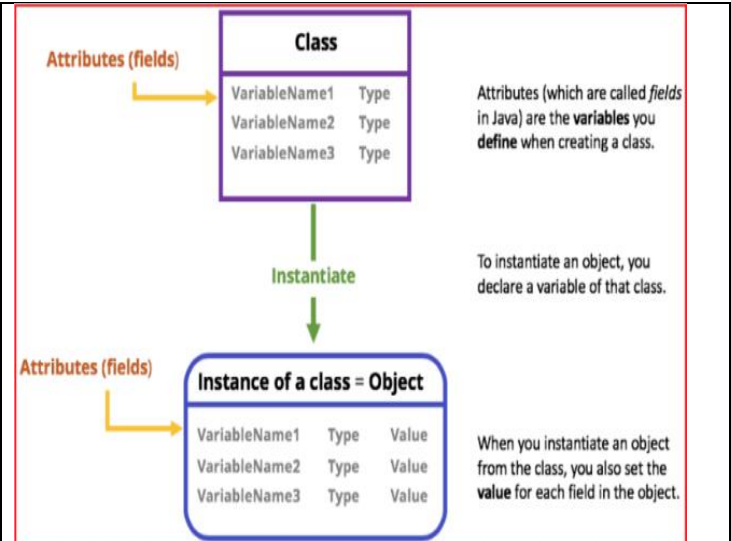


Class and Object

The Core Concept: Class and Object

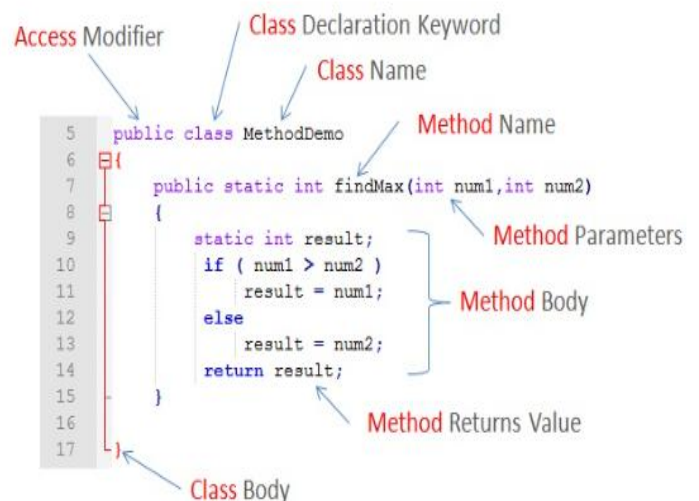
A Class is the Blueprint: It is the conceptual design, a template, or a factory mold. It does not physically exist; it just describes what something *will* look like and what it *will* do.

An Object is the Instance: It is the actual, physical item built directly from that blueprint. You can use one blueprint class to stamp out hundreds of individual objects

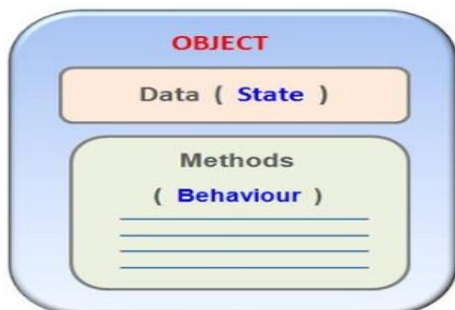


Anatomy of Class

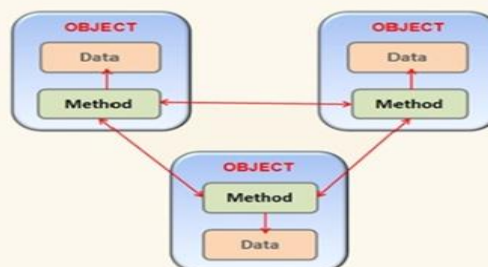
- **Attributes (Fields / Instance Variables):** What the object *knows* or remembers about itself (its data). Properties define the characteristics or attributes of an object.
- **Methods:** What the object can *do* (its functional actions).
- **Constructor:** The initialization engine that assigns starting values to an object's fields upon birth.



OOP - Concept Of Object



OPP - Program Organization



Example Blueprint

```
public class Car {
    // Attributes
    public String color;
    public int speed;

    // 1. THE CONSTRUCTOR (No return type, matches class name)
    public Car(String initialColor, int initialSpeed) {
        this.color = initialColor; // Sets the object's color to the incoming in
        this.speed = initialSpeed; // Sets the object's speed to the incoming in
    }

    // Method
    public void displayStatus() {
        System.out.println("The " + color + " car is going " + speed + " mph.");
    }
}
```

Instantiating: Making Objects from Your Class

new keyword in `main()` triggers the class constructor to build a live object in your computer's memory. instantiate and configure a object cleanly in a single line.

Dot Notation (.): The syntax syntax operator used to dig inside an object to access its variables or trigger its methods (e.g., `p1.name`).

```
// This automatically jumps to the constructor code above!
Car myCar = new Car("Red", 60);
myCar.displayStatus(); // Prints: The Red car is going 60 mph.
```

What Happens in Computer Memory During Instantiation?

When Java executes the line `Car genericCar = new Car("Red", 55);`, it performs three distinct steps under the hood:

1. **new Car(...):** Java allocates a brand-new chunk of blank memory big enough to hold a Car object.
2. **Car("Red", 55):** The constructor function wakes up, takes your values, and writes "Red" into the color slot and 55 into the speed slot inside that fresh memory space.
3. **Car genericCar = ...:** Java assigns a variable pointer named `genericCar` to reference (point directly to) that specific spot in memory so you can find it later using dot notation.

Encapsulation

In Java, access modifiers control the visibility and accessibility of your classes, methods, variables, and constructors. They are the foundation of **encapsulation**, which allows you to secure your data and hide the inner workings of your code.

Here is a quick overview of how the three main modifiers, plus the "default" behavior, restrict access:

Access Modifier	Inside Same Class	Inside Same Package	Subclasses (Different Package)	The World (Everywhere)
<code>public</code>	✓ Yes	✓ Yes	✓ Yes	✓ Yes
<code>protected</code>	✓ Yes	✓ Yes	✓ Yes (via inheritance)	✗ No
<i>default (no keyword)</i>	✓ Yes	✓ Yes	✗ No	✗ No
<code>private</code>	✓ Yes	✗ No	✗ No	✗ No

Public (`public`)

The public modifier provides the **least restrictive** level of access. Anything declared as public can be seen and used by any other class in your entire project. [1, 2]

- **When to use:** Use this for your main API endpoints, major application services, helper utilities, or anything designed to be shared globally across packages.

Private (`private`)

The private modifier provides the **highest level of restriction**. Variables or methods marked private are hidden from every other class in your project—even classes inside the exact same folder (package). They can only be accessed within the specific class where they are defined. [1, 2, 3]

- **When to use:** Use this by default for almost all of your class variables (fields). You then expose them safely using public "getter" and "setter" methods if other classes need to read or change them.

Protected (`protected`)

The protected modifier is tailored specifically for inheritance. It allows a parent class to share its components with its children (subclasses), even if those child classes are located in completely different folders or packages. Additionally, any *other* class inside the same package can also see it.

- **When to use:** Use this when building a framework or class hierarchy where child classes genuinely need direct access to a parent's methods or data to extend its functionality.

Note on "Default" (Package-Private)

If you do not type public, private, or protected next to a variable or method, Java automatically assigns it **default** access. This means the code is visible to everything inside the **same package**, but entirely hidden from the outside world. [1, 2, 3]

Would you like to see a practical example of **how these modifiers interact** across different packages, or do you want to explore how **getters and setters** protect private fields? [1]

Here is a complete, working example

Using two different packages (animals and zoo) to show exactly how all four access levels behave across your project

Package 1: animals

First, let's create a base class named Animal that defines a variable for each of the four access levels. And look another class inside that **same package** to see what it can access.

```
package animals;

public class Animal {
    // 1. Public: Accessible from anywhere in the project
    public String publicName = "Global Lion";

    // 2. Protected: Accessible in the same package AND by subclasses
    protected String protectedSound = "Roar";
    // 3. Default (No modifier): Accessible ONLY in the same package

    String defaultColor = "Golden";
    // 4. Private: Accessible ONLY inside this Animal class

    private String privateId = "ID-9942";

    public void testInsideClass() {
        // All of these work perfectly inside the same class
        System.out.println(publicName); // ✅ Success
        System.out.println(protectedSound); // ✅ Success
        System.out.println(defaultColor); // ✅ Success
        System.out.println(privateId); // ✅ Success
    }
}
```

```
// ● The Getter Method: Anyone can call this to READ the ID
public String getPrivateld() {
    return this.privateld;
}

package animals;

public class PackageCompanion {

    public void testSamePackage() {
        Animal animal = new Animal();
        System.out.println(animal.publicName); // ✓ Success
        System.out.println(animal.protectedSound); // ✓ Success
        System.out.println(animal.defaultColor); // ✓ Success

        // System.out.println(animal.privateld); // ✗ COMPILE ERROR: privateld is private
        // to Animal
    }
}
```

Package 2: zoo

Now, let's move outside the animals package to see how inheritance and external access restrictions work.

```
package zoo;
import animals.Animal; // Must import since it's a different package

public class Lion extends Animal {
    public void testSubclass() {
        // 1. Public works everywhere
        System.out.println(publicName); // ✓ Success

        // 2. Protected works because Lion EXTENDS Animal (Inheritance)
        System.out.println(protectedSound); // ✓ Success
    }
}
```

```
// 3. Default fails because Lion is in a DIFFERENT package
// System.out.println(defaultColor); // ✗ COMPILE ERROR: Not visible outside package
// 4. Private fails because it belongs exclusively to Animal
// System.out.println(privateld); // ✗ COMPILE ERROR: Not visible outside Animal
}
```

Run the code and see how the modifiers react.

This Main class is in a completely separate package named mainrunner to simulate "the rest of the world" looking into your code.

```
package mainrunner;

import animals.Animal; // Import the base class
import zoo.Lion; // Import the subclass

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Testing from an unrelated Main Class ---");
        Animal genericAnimal = new Animal();

        // 1. Public works everywhere
        System.out.println("Public Field: " + genericAnimal.publicName); // ✓ Success

        // 2. Protected fails here (Main is not a subclass of Animal)
        // System.out.println(genericAnimal.protectedSound); // ✗ COMPILE ERROR

        // 3. Default fails here (Main is in a different package)
        // System.out.println(genericAnimal.defaultColor); // ✗ COMPILE ERROR

        // 4. Private fails completely
        // System.out.println(genericAnimal.privateld); // ✗ COMPILE ERROR

        System.out.println("\n--- Testing via the Lion Subclass ---");
        Lion simba = new Lion();
    }
}
```

```
// This triggers the method inside Lion, which can safely
// read the public and protected fields due to inheritance
simba.testSubclass();
}
}
```

What Happens When You Run This?

If you uncomment any of the red-marked lines (✗), your code will refuse to compile. Left as it is above, it compiles flawlessly and prints out:

```
text
--- Testing from an unrelated Main Class ---
Public Field: Global Lion

--- Testing via the Lion Subclass ---
Global Lion
Roar
```

Getter and Setter method

A **getter method** is a public method used to safely retrieve (get) the value of a hidden, private variable from outside its class.

By making class variables private, you prevent outside classes from accidentally changing or corrupting your data. The getter method acts as a controlled, **read-only window** to the world.

```
package mainrunner;
import animals.Animal;

public class Main {
    public static void main(String[] args) {
        Animal genericAnimal = new Animal();

        // ❌ This still causes a compile error:
        // System.out.println(genericAnimal.privateId);

        // ✅ This works perfectly!
        String idValue = genericAnimal.getPrivateId();
        System.out.println("Accessed via Getter: " + idValue);
    }
}
```

Why do this instead of just making the variable public?

1. **Read-Only Protection:** By providing only a getter (and no setter), you make the variable completely immutable to the outside world. Other classes can see it, but they can **never change it**.
2. **Data Logging & Control:** You can add logic inside the getter. For example, you could log a message every time a sensitive piece of data is accessed:

A **setter method** is a public method used to safely update or change (set) the value of a hidden, private variable from outside its class.

While getters provide read-only access, setters provide **write-only access**. By routing all changes through a setter method, your class can inspect, validate, or reject new values before updating the actual variable.

Why use a Setter instead of a public variable?

- **Data Validation:** You can block bad inputs (like preventing a `setAge(int age)` method from accepting negative numbers).
- **Encapsulation:** The internal storage logic is hidden. If you later decide to change how IDs are structured (e.g., adding a prefix automatically), you only change the code *inside* the setter, without breaking anyone else's code.

```
package animals;
public class Animal {
    .....
    // 🟢 The Setter Method: Anyone can call this to CHANGE the ID
    public void setPrivateId(String newId) {
        // Validation: Ensure the new ID isn't empty or null
        if (newId != null && !newId.trim().isEmpty()) {
            this.privateId = newId; // 'this.privateId' refers to the private field above
        } else {
            System.out.println("Error: Invalid ID provided. Update rejected.");
        }
    }
}
```

```
package mainrunner;
import animals.Animal;
public class Main {
    public static void main(String[] args) {
        Animal genericAnimal = new Animal();
        // ❌ This causes a compile error:
        // genericAnimal.privateId = "NEW-ID-123";

        // ✅ This works perfectly!
        genericAnimal.setPrivateId("NEW-ID-123");

        // ❌ This will trigger our validation rule and keep the old ID
        genericAnimal.setPrivateId("");
    }
}
```

This keyword

How **this keyword** works inside getters and [setters](#)

In Java, the **this keyword** is a reference variable that refers directly to the **current object instance** you are working inside.

Inside getters and setters, its primary job is to **resolve naming conflicts** (called variable shadowing) and clarify exactly which variable you mean.

The Problem: Variable Shadowing

When you write a setter method, you usually name the incoming parameter the exact same name as the class variable.

<pre>public class Animal { private String name; // 1. Instance variable (Class level) public void setName(String name) { // 2. Local parameter (Method level) // If you write: name = name;</pre>	Without this, Java gets confused. It assumes both name references mean the local parameter (number 2) because it is closer in scope. This means the code effectively does nothing—it assigns the parameter to itself, leaving the actual class variable blank. This is called variable shadowing.
<pre>public class Animal { private String name; // ● Inside a Setter public void setName(String name) { this.name = name; // "this.name" is the class variable. "name" is the parameter. } // ● Inside a Getter public String getName() { return this.name; // Refers directly to the class variable } }</pre>	The Solution: Using this To fix this, you prepend this. to the class variable. This explicitly tells Java: <i>"Hey, set the class-level variable belonging to this specific object equal to the local parameter value."</i>

Visualizing this in Memory

Imagine you create two separate Animal objects in your Main class:

```
Animal dog = new Animal();  
Animal cat = new Animal();  
  
dog.setName("Buddy"); // Inside setName(), 'this' points to 'dog'  
cat.setName("Whiskers"); // Inside setName(), 'this' points to 'cat'
```

When `dog.setName("Buddy")` runs:

- this automatically binds itself to the dog object.
- `this.name = name;` translates behind the scenes to: `dog.name = "Buddy";`

When `cat.setName("Whiskers")` runs:

- this automatically binds itself to the cat object.
- `this.name = name;` translates behind the scenes to: `cat.name = "Whiskers";`

Is this mandatory in Getters?

No. In the `getName()` method above, there is no parameter named `name` causing a conflict. Therefore, writing `return name;` works perfectly fine. However, many developers still write `return this.name;` because it makes it immediately obvious to anyone reading the code that `name` is a class-level variable.

Inheritance

Inheritance is one of the pillars of Object-Oriented Programming. It allows a new class to inherit (absorb) all the attributes and methods of an existing class automatically.

Think of inheritance like **genetic traits**. A child inherits traits from their parent, but the child can also have unique traits of their own.

<pre> classDiagram class Car { - color - speed + displayStatus() } class ElectricCar { - batteryLevel + chargeBattery() } Car < -- ElectricCar </pre> PARENT: Car ← Contains general car logic - color, speed + displayStatus() extends (Inherits everything) CHILD: ElectricCar ← Automatically gets color, speed, and disp - batteryLevel ← Added unique attribute + chargeBattery() ← Added unique method	• Parent Class (Superclass): The general, base class that holds common traits (e.g., Car). • Child Class (Subclass): The specialized class that inherits everything from the parent but adds unique twists (e.g., an ElectricCar). • The Keyword: extends In Java, you use the extends keyword to establish an inheritance relationship. • @Override Annotation: You place this helper tag right above the child method. It tells Java to double-check your code to ensure you didn't accidentally typo the name or parameters.
--	---

Why Use Inheritance?

- **Code Reusability:** You don't have to rewrite code for color, speed, or displayStatus() if you decide to build a GasCar, a Truck, or a RaceCar later. You just extend Car.
 - While inheritance gives the child class all the parent's methods automatically, sometimes the default parent behavior isn't quite right for the child. **Method Overriding** occurs when a child class rewrites a method it inherited from its parent class to give it a new, specialized behavior. Child keep the exact same method name and inputs, but change what happens inside the execution brackets { }.
- ❓ **Same Name:** The method name in the child class must exactly match the parent class.
- ❓ **Same Parameters:** The inputs (arguments) must match exactly.

```

public class ElectricCar extends Car {
    public int batteryLevel;

    public ElectricCar(String color, int speed, int battery) {
        super(color, speed);
        this.batteryLevel = battery;
    }

    // — METHOD OVERRIDING HAPPENS HERE —
    @Override
    public void displayStatus() {
        // We redefine the action specifically for electric cars
        System.out.println("The " + color + " electric car is going " + speed +
            " mph with " + batteryLevel + "% battery remaining.");
    }
}

```

The `super` Shortcut in Overriding

Notice that inside the child's `displayStatus()`, we re-wrote the part printing the color and speed. If the parent method has a ton of complex code, re-writing it is a waste of time.

You can use `super.methodName()` to run the parent's version of the method *first*, and then add your child logic right below it: 

java

```
@Override
public void displayStatus() {
    super.displayStatus(); // Runs the parent version (Prints: The White car is !
    System.out.println("Battery Level: " + batteryLevel + "%"); // Adds unique cl
}
```

```
public class Main {
    public static void main(String[] args) {

        Car basicCar = new Car("Red", 55);
        ElectricCar tesla = new ElectricCar("White", 70, 85);

        System.out.println("--- Testing Parent Method ---");
        basicCar.displayStatus();
        // Prints: The Red car is going 55 mph.

        System.out.println("\n--- Testing Overridden Child Method ---");
        tesla.displayStatus();
        // Prints: The White electric car is going 70 mph with 85% battery remaini

    }
}
```

- Java is smart under the hood. Even though both objects are used. `displayStatus()`, Java looks at the specific object type at runtime and executes the customized version for the `ElectricCar`.
- If not `Override`, Java fall back to base class function.

Abstract Class

The Rules of Abstract Methods

1. **No Body:** Abstract methods end with a semicolon ; instead of curly braces {}.
2. **The abstract Keyword:** You must place the word abstract before the method name.
3. **Abstract Class Requirement:** If a class contains even *one* abstract method, **the entire class must be declared as abstract**. You cannot instantiate (create a new) abstract class directly.
4. **Child Obligation:** Any normal child class that inherits from an abstract parent **must override and write the body** for all abstract methods, or your code will fail to compile.

The Abstract Parent Class

Traditional gas car accelerates by burning fuel, while an electric car accelerates by pulling power from a battery pack. Instead of writing a generic, useless default method in the parent class, we make it **abstract**.

```
// We add 'abstract' to the class because it contains an abstract method
public abstract class Vehicle {
    public String color;

    public Vehicle(String color) {
        this.color = color;
    }

    // Standard method: Inherited normally by all children
    public void honk() {
        System.out.println("Beep beep!");
    }

    // ABSTRACT METHOD: No curly braces! Just a rule for the children.
    public abstract void accelerate();
}
```

The Child Classes (Fulfilling the Contract)

Now, any child class that extends Vehicle is forced by Java to implement accelerate(). If you forget to add it, Java will give you a compilation error.

```
public class GasCar extends Vehicle {
    public GasCar(String color) {
        super(color);
    }

    // We MUST override and provide a body for the abstract method
    @Override
    public void accelerate() {
        System.out.println("The gas engine revs! Fuel is injected into cylinders");
    }
}

public class ElectricCar extends Vehicle {
    public ElectricCar(String color) {
        super(color);
    }

    // We MUST override and provide a body for the abstract method too
    @Override
    public void accelerate() {
        System.out.println("The electric motor hums instantly! Battery power surges");
    }
}
```

Running the Code in **Main**

Remember, you **cannot** do `Vehicle v = new Vehicle("Red");` because abstract classes are incomplete blueprints. You can only instantiate the concrete children.

```
public class Main {  
    public static void main(String[] args) {  
  
        // Instantiate the concrete child classes  
        GasCar mustang = new GasCar("Blue");  
        ElectricCar tesla = new ElectricCar("White");  
  
        // Both can use the normal inherited method  
        mustang.honk(); // Prints: Beep beep!  
  
        // Each executes its own mandatory version of the abstract method  
        mustang.accelerate(); // Prints: The gas engine revs!...  
        tesla.accelerate();   // Prints: The electric motor hums!...  
    }  
}
```

Why Use Abstract Methods?

Abstract methods allow you to enforce a standard design structure across multiple different classes. It guarantees that no matter what kind of vehicle someone programs in the future, it is guaranteed to have an `accelerate()` action built into it.

Polymorphism

Polymorphism means "many forms." In Java, it allows you to treat different child objects (`GasCar` , `ElectricCar`) as if they were their parent type (`Vehicle`) [sports]. This is powerful because it lets you write one piece of code that works automatically with any type of vehicle you create in the future. 

The Big Magic: Upcasting

Polymorphism allows you to declare a variable using the **Parent type**, but instantiate it using a **Child type**. 

java

```
// Declared as a Vehicle, but instantiated as a GasCar
Vehicle myRide = new GasCar("Blue");
```

Even though the variable type on the left is `Vehicle`, Java looks at the real object on the right (`GasCar`) at runtime and executes the `GasCar` version of your methods.

Polymorphism in Action (Code Example)

Let's look at how polymorphism allows us to manage all our cars inside a single array loop and pass them into a single function.

Why is this so useful?

Imagine you are building a video game with 100 different types of cars. Without polymorphism, you would have to write 100 separate functions to start a race (`startGasCarRace()`, `startElectricCarRace()`, etc.). With polymorphism, you write **one single function** that accepts `Vehicle`, and it handles all 100 car types perfectly.

What Prints to the Console?

```
--- Testing Loop Polymorphism ---
The gas engine revs! Fuel is injected into cylinders.
The electric motor hums instantly! Battery power surges.

--- Testing Function Polymorphism ---
The Red vehicle prepares... The gas engine revs! Fuel is injected into cylinders.
The Black vehicle prepares... The electric motor hums instantly! Battery power surges.
```

Polymorphism in Action (Code Example)

```

public class Main {
    public static void main(String[] args) {

        // 1. POLYMORPHISM IN ARRAYS
        // We create an array of type Vehicle. It can hold ANY child of Vehicle!
        Vehicle[] fleet = new Vehicle[2];

        fleet[0] = new GasCar("Blue");    // Upcasted automatically
        fleet[1] = new ElectricCar("White"); // Upcasted automatically

        System.out.println("--- Testing Loop Polymorphism ---");
        for (Vehicle v : fleet) {
            // Java automatically figures out which version of accelerate() to run!
            v.accelerate();
        }

        System.out.println("\n--- Testing Function Polymorphism ---");
        GasCar mustang = new GasCar("Red");
        ElectricCar tesla = new ElectricCar("Black");

        // 2. POLYMORPHISM IN FUNCTIONS
        // We can pass different object types into the exact same function
        startRace(mustang);
        startRace(tesla);
    }

    // This function accepts a generic Vehicle parent type
    public static void startRace(Vehicle v) {
        System.out.print("The " + v.color + " vehicle prepares... ");
        v.accelerate(); // Polymorphic method call
    }
}

```

Variable scope

Different types of [variables](#) are

- Local Variables
- Instance Variables
- Static Variables

Local Variables A variable defined within a block or method or constructor is called a local variable. • These variables are created when the block is entered, or the function is called and destroyed after exiting from the block or when the call returns from the function. • The scope of these variables exists only within the block in which the variables are declared, i.e., we can access these variables only within that block. • Initialization of the local variable is mandatory before using it in the defined scope.	<pre> Class hcl guvi { public static void main(String[] args) { // Declared a Local Variable int var = 10; // This variable is local to this main method only System.out.println("Local Variable: " + var); } } </pre>
Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block. • As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed. • Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used. • Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable. • Instance variables can be accessed only by creating objects. • We initialize instance variables using constructors while creating an object. We can also use instance blocks to initialize the instance variables.	<pre> // Java Program to demonstrate class hcl guvi { // Declared instance variable String student = "Shubham Jain"; public static void main(String[] args) { HCL Guvi Zen=new HCLGuvi(); // geek variable can be accessed by creating object System.out.print(Zen. student); } } </pre>

Static Variables

Static variables are also known as class variables.

- These variables are declared similarly to instance variables. The difference is that static variables are declared using the static keyword within a class outside of any method, constructor, or block.
- Unlike instance variables, we can only have one copy of a static variable per class, irrespective of how many objects we create.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Initialization of a static variable is not mandatory. Its default value is dependent on the data type of variable

```
// Java Program to demonstrate class
hcl guvi {
// Declared instance variable Public
static String student = "Shubham Jain";
public static void main(String[] args)
{
// student variable can be accessed without
object
// static variable
System.out.print(hclguvi. student);
}
}
```

Scope of variables

Variable Type	Scope	Lifetime
Instance variable	Troughout the class except in static methods	Until the object is available in the memory
Class variable	Troughout the class	Until the end of the program
Local variable	Within the block in which it is declared	Until the control leaves the block in which it is declared

Project - Build a Student Grade & Honor Roll Tracker

This project bridges your fundamental knowledge (I/O, logic, and functions) with the next set of essential Java concepts: **Strings**, **Arrays**, **Wrapper Classes/Type Casting**, and **Basic Classes/Objects**, inheritance and Polymorphism.

The Goal

Build an application that reads a student's raw data string from user.

Extracts their numeric grades into an array, calculates their average, and generates an automated report profile card.

This UnifiedTracker.java step-by-step evolution of GradeTracker application.

Task 1: Read messy Student Names from user

- Concept: Strings and String Manipulation (.trim(), .substring(), .toUpperCase())
- Goal: Capture a messy text string from the keyboard, clean it, and format it properly.

```
import java.util.Scanner;

public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // Clean and format text
        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);

        System.out.println("Saved Name: '" + formattedName + "'");

        // [Task 2 code will be inserted directly here next]
        inputScanner.close();
    }
}
```

Step B: Use the Object inside your Main Runner program

Update your main running file (GradeTracker.java) to make a live instance from your blueprint:

```
public class GradeTracker {
    public static void main(String[] args) {

        // 1. Prepare data variables
        String processedName = "Alex Smith";
        int[] scores = {95, 88, 91, 94};

        // 2. OOP Concept: Instantiate a real Student object
        container from our blueprint
        Student currentStudent = new Student(processedName,
        scores);

        // 3. Ask the object to execute its embedded logic method
        System.out.println("=== Final Report Card ===");
        System.out.println("Student Profile Name: " +
        currentStudent.name);

        if (currentStudent.qualifiesForHonorRoll()) {
            System.out.println("Status: CONGRATULATIONS! Placed
            on the Academic Honor Roll.");
        } else {
            System.out.println("Status: Satisfactory standing.");
        }
    }
}
```

Task 2: Request user to input Math and Science grade and calculate the average

New Concepts: Type Casting & Wrapper Classes (Integer.parseInt())

Parse the use given math integers, and use type casting for a precise decimal average.

```
import java.util.Scanner;

public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
        System.out.println("Saved Name: '" + formattedName + "'");

        // — ADDING TASK 2 NEW CODE DIRECTLY BELOW —
        System.out.println("\n=== Task 2: Parsing & Casting ===");
        System.out.print("Enter text grade for Math: ");
        String mathText = inputScanner.next();
        System.out.print("Enter text grade for Science: ");
        String scienceText = inputScanner.next();

        // Wrapper parsing
        int mathScore = Integer.parseInt(mathText);
        int scienceScore = Integer.parseInt(scienceText);

        // Type casting for decimal average
        double initialAverage = (double) (mathScore + scienceScore) / 2;
        System.out.println("Initial 2-Class Average: " + initialAverage + "%");

        // [Task 3 code will be inserted directly here next]
        inputScanner.close();
    }
}
```

Task 3: Scale Up to a Full Schedule.

- Concept: (Arrays & Loops)
- Goal: Instead of calculating just Math and Science as loose variables.
Ask how many total classes they have, create an **Array** of that size, and use a for loop to capture everything dynamically.

```
import java.util.Scanner;

public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
        System.out.println("Saved Name: '" + formattedName + "'");

        System.out.println("\n=== Task 2: Parsing & Casting ===");
        System.out.print("Enter text grade for Math: ");
        String mathText = inputScanner.next();
        System.out.print("Enter text grade for Science: ");
        String scienceText = inputScanner.next();

        int mathScore = Integer.parseInt(mathText);
        int scienceScore = Integer.parseInt(scienceText);

        double initialAverage = (double) (mathScore + scienceScore) / 2;
        System.out.println("Initial 2-Class Average: " + initialAverage + "%");

        // — ADDING TASK 3 NEW CODE DIRECTLY BELOW —
        System.out.println("\n=== Task 3: Arrays & Loops ===");
        System.out.print("How many total classes do you want to track in the array? ");
        int totalClasses = inputScanner.nextInt();

        int[] scheduleGrades = new int[totalClasses];
        int sum = 0;

        for (int i = 0; i < scheduleGrades.length; i++) {
            System.out.print("Enter text grade for class #" + (i + 1) + ": ");
            String textGrade = inputScanner.next();
            // We combine Task 2's Wrapper parsing directly inside our array loop!
            scheduleGrades[i] = Integer.parseInt(textGrade);
            sum += scheduleGrades[i];
        }

        double finalAverage = (double) sum / scheduleGrades.length;
        System.out.println("Final Array Schedule Average: " + finalAverage + "%");

        // [Task 4, 5, and 6 will now pull this data into an Object architecture below]
        inputScanner.close();
    }
}
```

Task 4: Clean the Main Architecture (Introducing Multiple Functions)

Introduce **4 helper functions** outside of `main`. We then update `main` to pass its data straight into these functions, keeping the execution flow clean and readable.

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

public class UnifiedTracker {

    // — TASK 4: FUNCTION 1 (Extracts Text Cleaning Logic from Task 1) —
    public static String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    // — TASK 4: FUNCTION 2 (Extracts Parsing & Casting Logic from Task 2) —
    public static double parseAndCalculateInitial(String textGrade1, String textGrade2) {
        int score1 = Integer.parseInt(textGrade1);
        int score2 = Integer.parseInt(textGrade2);
        return (double) (score1 + score2) / 2;
    }

    // — TASK 4: FUNCTION 3 (Extracts Array Loop Math Logic from Task 3) —
    public static double calculateAverage(int[] gradesArray) {
        int sum = 0;
        for (int score : gradesArray) {
            sum += score;
        }
        return (double) sum / gradesArray.length;
    }

    // — NEW TASK 4: FUNCTION 4 (Replaces and Handles the Task 3 For-Loop Input) —
    public static void fillGradesArray(Scanner inputScanner, int[] scheduleGrades) {
        for (int i = 0; i < scheduleGrades.length; i++) {
            System.out.print("Enter text grade for class #" + (i + 1) + ": ");
            String textGrade = inputScanner.next();
            // Performs the Task 2 Wrapper parsing directly into the array element
            scheduleGrades[i] = Integer.parseInt(textGrade);
        }
    }
}
```

```

// — CLEAN MAIN EXECUTIVE RUNNER —
public static void main(String[] args) {
    Scanner inputScanner = new Scanner(System.in);

    // --- TASK 1 CUMULATIVE REUSE ---
    System.out.println("=== Task 1: String Input & Cleaning ===");
    System.out.print("Enter student name (e.g., ' john doe '): ");
    String rawName = inputScanner.nextLine();

    String formattedName = formatName(rawName); // Function Call 1
    System.out.println("Saved Name: '" + formattedName + "'");

    // --- TASK 2 CUMULATIVE REUSE ---
    System.out.println("\n=== Task 2: Parsing & Casting ===");
    System.out.print("Enter text grade for Math: ");
    String mathText = inputScanner.next();
    System.out.print("Enter text grade for Science: ");
    String scienceText = inputScanner.next();

    double initialAverage = parseAndCalculateInitial(mathText, scienceText); // Function Call 2
    System.out.println("Initial 2-Class Average: " + initialAverage + "%");

    // --- TASK 3 CUMULATIVE REUSE (With Loop Replaced) ---
    System.out.println("\n=== Task 3 & 4: Array Size & Loop Function ===");
    System.out.print("How many total classes do you want to track? ");
    int totalClasses = inputScanner.nextInt();

    // Declare the blank fixed-size array container
    int[] scheduleGrades = new int[totalClasses];

    // Function Call 4: Loops and populates the scheduleGrades array from inside the function
    fillGradesArray(inputScanner, scheduleGrades);

    // --- TASK 4 OUTPUT VERIFICATION ---
    System.out.println("\n=== Task 4: Final Calculated Return Values ===");
    double computedAverage = calculateAverage(scheduleGrades); // Function Call 3
    System.out.println("Clean Function Calculated Average: " + computedAverage + "%");

    inputScanner.close();
}
}

```

Task 5: OOPs concepts – class, encapsulation

- Concept: OOP - Encapsulation
- Goal: Instead of loose data and method. Create a Class Student with

Data:	Student name, grades
Subject list	static String[] containing your subject names: Math, Science, Social, English, and French
Constructor	In Constructor, ask user to input grades for above subjects.
Private Method	fillGradesArray() Instead of asking the user for a raw total number of classes, the program automatically loops through these specific subjects, prompts for each one by name, and maps them to their respective positions in the array.
Private Method	Instead of formatting in Class Main. Format here. private String formatName(String rawName)
Public Method	calculateAverage() Loop through data: grades[] and calculate grade.

```
// — MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- DATA COLLECTION INTAKE ---
        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // --- OBJECT BIRTH SYNTHESIS ---
        // We instantiate the object. It handles text cleaning and sequential
        // subject tracking completely on its own inside the constructor!
        Student activeStudent = new Student(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n=====");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeStudent.name);
        System.out.println("Total Classes:   " + activeStudent.grades.length);

        // Call instance method to compute arithmetic output
        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:   " + computedAverage + "%");
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — FULLY ENCAPSULATED OBJECT BLUEPRINT —
class Student {
    // Shared structural template for subjects
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    // Attributes
    public String name;
    public int[] grades;

    // — REFACTORED AUTOMATED CONSTRUCTOR —
    public Student(String rawName, Scanner inputScanner) {
        // 1. Call private helper to format text upon birth
        this.name = formatName(rawName);

        // 2. Initialize internal array capacity to match the total number of subjects
        this.grades = new int[SUBJECTS.length];

        // 3. Call private helper to run subject-by-subject loops and error safeguards
        fillGradesArray(inputScanner);
    }

    // — PRIVATE INSTANCE HELPER METHODS —

    // Function 1: Pure internal text processing (Private, instance method)
    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    // Function 2: Enumerates through subject strings directly into 'this.grades'
    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            // Prompts specifically by the subject name
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();

            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("❌ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    // Function 3: Mathematical engine targeting 'this.grades' automatically
    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```

Task 6: OOPs concepts – Data hiding

Now that your fundamental structural framework is locked down, let me know if you would like to proceed with **The Data Hiding Step**: Converting public String name to private String name and creating an official **Getter Method** (public String getName()) to implement proper professional Java security standards.

- **Private Fields:** name and grades fields are changed to private. The main method can no longer access activeStudent.name or activeStudent.grades.length directly without causing a compilation error.
- **Getter Methods:** We introduce getName() and getGrades() to allow controlled, read-only access to our private fields. By using **Getters**, your main program gets the variables it needs, but the object maintains absolute authority over its internal state.
- **Zero Code Discarded:** The fixed SUBJECTS array layout, error-trapping inputs, and formatting mechanics remain fully intact

```
// — MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // Instantiating the object profile
        Student activeStudent = new Student(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n=====");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("=====");

        // — UPGRADED ACCESS VIA GETTER METHOD CALLS —
        // Writing activeStudent.name here would now cause a Java compilation error!
        System.out.println("Student Profile: " + activeStudent.getName());
        System.out.println("Total Classes:   " + activeStudent.getGrades().length);

        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:   " + computedAverage + "%");
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```

// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — FULLY ENCAPSULATED & HIDDEN OBJECT BLUEPRINT —
class Student {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    // — CONCEPT UPGRADE: CHANGED FROM PUBLIC TO PRIVATE (DATA HIDING) —
    private String name;
    private int[] grades;

    // Automated Constructor (Unchanged)
    public Student(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    // — NEW PUBLIC GETTER METHODS (THE DOORWAYS TO READ HIDDEN DATA) —

    // Allows outside classes to read the hidden name safely
    public String getName() {
        return this.name;
    }

    // Allows outside classes to read the hidden grades list size or values safely
    public int[] getGrades() {
        return this.grades;
    }

    // — PRIVATE INSTANCE HELPER METHODS (Fully Retained) —

    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }
}

```

```

private void fillGradesArray(Scanner inputScanner) {
    System.out.println("\n--- Entering Grades for Class Schedule ---");
    for (int i = 0; i < this.grades.length; i++) {
        System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
        String textGrade = inputScanner.next();

        try {
            this.grades[i] = Integer.parseInt(textGrade);
        } catch (NumberFormatException e) {
            System.out.println("✗ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
            this.grades[i] = 0;
        }
    }
}

public double calculateAverage() {
    int sum = 0;
    for (int score : this.grades) {
        sum += score;
    }
    return (double) sum / this.grades.length;
}

```

Task 7: OOPs – Inheritance and overriding

- Create a child class called **GraduateStudent** that extends your base Student. This allows the child class to automatically reuse the fields, getters, array logic, and validation rules from the parent class without rewriting a single line of that code.
- Add Unique Child Traits: We add a new private attribute unique to graduate students: private String thesisTitle, along with its own public getter (getThesisTitle()).
- introduce an status metric called **qualifiesForHonorRoll()** inside the parent class, which checks if a student has an average of **90.0% or higher**. Then, using **Method Overriding**, the GraduateStudent child class will rewrite this exact method to enforce a stricter rule: graduate students must maintain a **95.0% or higher** to make the honor roll.

```
// — MAIN RUNNER EXECUTIVE SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Graduate Student Data Entry ===");
        System.out.print("Enter raw student name (e.g., ' mariah carey '): ");
        String rawName = inputScanner.nextLine();

        System.out.print("Enter Research Thesis Topic: ");
        String thesis = inputScanner.nextLine();

        // Child Object Birth
        GraduateStudent activeGrad = new GraduateStudent(rawName, thesis, inputScanner);

        // --- REPORT GENERATION ---
        System.out.println("\n=====");
        System.out.println("      GRADUATE ACADEMIC REPORT      ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeGrad.getName());
        System.out.println("Total Classes:   " + activeGrad.getGrades().length);
        System.out.println("Research Focus:  " + activeGrad.getThesisTitle());

        double avg = activeGrad.calculateAverage();
        System.out.println("Final Average:   " + avg + "%");

        // — EVALUATING THE OVERRIDDEN METHOD —
        // Java executes the customized child method logic rules automatically!
        if (activeGrad.qualifiesForHonorRoll()) {
            System.out.println("Honor Roll:      ★ APPROVED (95%+ Target Cleared) ★");
        } else {
            System.out.println("Honor Roll:      Standard Graduate Standing");
        }
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```
// — INHERITANCE (CHILD CLASS) —
class GraduateStudent extends Student {

    private String thesisTitle;

    // Child Constructor calling parent super
    public GraduateStudent(String rawName, String thesisTitle, Scanner inputScanner) {
        super(rawName, inputScanner);
        this.thesisTitle = thesisTitle;
    }

    public String getThesisTitle() {
        return this.thesisTitle;
    }

    // — NEW STEP: METHOD OVERRIDING —
    @Override
    public boolean qualifiesForHonorRoll() {
        // Graduate students must hit a tougher threshold (95.0%)
        // Notice it calls the inherited calculateAverage() method seamlessly!
        return calculateAverage() >= 95.0;
    }
}
```

- **Parent Method:** We add public boolean qualifiesForHonorRoll() inside the Student class with a base threshold of >= 90.0.
- **Method Overriding:** We copy the exact same method signature inside GraduateStudent, use the **@Override** annotation tag, and change the logic check to >= 95.0.

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — PARENT CLASS BLUEPRINT (Fully Encapsulated) —
class Student {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    private String name;
    private int[] grades;

    // Automated Parent Constructor
    public Student(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    // Public Getters
    public String getName() {
        return this.name;
    }

    public int[] getGrades() {
        return this.grades;
    }

    // Baseline Honor Roll Rule (90.0% or higher)
    public boolean qualifiesForHonorRoll() {
        return calculateAverage() >= 90.0;
    }

    // Private Instance Helpers
    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();

            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("✗ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```

Task 8: OOPS – Polymorphism

Instead of creating separate, rigid code streams for each student type, polymorphism lets you treat both regular Student objects and GraduateStudent objects as if they belong to the same parent type Student.

- Adjust our main method to instantiate a mix of profiles and store them inside a single **Parent Array**. We will then use a single loop to scan the directory, and Java will automatically figure out which honor roll logic rules to apply to each student at runtime.
- We have wrapped the entire application inside an infinite loop (**while (true)**), which allows it to run dynamically for as many students as you want. At the start of each turn, the program asks if you want to add a **Regular Student**, a **Graduate Student**, or **Exit** the system. If you choose to exit, the program safely terminates the loop using the break keyword.

What Changes in This Step:

1. Dynamic Program Loop (while (true)): The application runs continuously until explicitly stopped.
2. User-Driven Object Decisions: An interactive selection menu lets the user choose the specific object type to manufacture on the fly.
3. **Parent Polymorphic Array**: We build a directory array using the parent blueprint type:

```
Student[] schoolRoll = new Student[2];
```
4. **Dynamic Slots**: Slot 0 is assigned a regular Student object, and slot 1 is assigned a GraduateStudent child object. Java allows this because a graduate student *is* a student. [1]
5. **Runtime Magic**: Inside our processing loop, the exact same method call (current.qualifiesForHonorRoll()) switches its underlying engine automatically—applying the 90.0% rule to slot 0 and the stricter 95.0% rule to slot 1 break: Instantly destroys the infinite loop the millisecond choice 3 is entered.
6. continue: Safely throws the application back to the menu start if the user inputs an unexpected integer.
7. instanceof: A new helper keyword used at the end to check: "Is the student currently passing through the engine a graduate student?" If yes, it safely unlocks and displays their hidden thesis title.

This seems like production-grade console application! Congratulation on First project.

Note:

No changes to class Student and class GraduateStudent for adding polymorphism.

Only adjust main method to instantiate a mix of profiles and store them inside a single **Parent Array**.

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println("  WELCOME TO THE GLOBAL STUDENT ENGINE  ");
        System.out.println("=====");

        // — DYNAMIC CONTINUOUS WHILE LOOP —
        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear out leftover newline token from scanner buffer

            // — DYNAMIC EXIT JUMP CONDITION —
            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break; // Instantly breaks the infinite while loop to end the program safely
            }

            // Route user choices or intercept bad choices
            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("✗ Invalid option. Please select 1, 2, or 3.");
                continue; // Skips the rest of this iteration and restarts the menu loop
            }

            // GATHER SHARED BASE PROFILE DETAILS
            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            // Declare a generic parent type pointer variable to handle polymorphism seamlessly
            Student currentStudent = null;

            // — USER DECIDES THE INSTANTIATION TYPE —
            if (menuChoice == 1) {
                // Instantiates a regular student profile
                currentStudent = new Student(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                // Instantiates a specialized graduate student child profile
                currentStudent = new GraduateStudent(rawName, thesis, inputScanner);
            }
        }
    }
}
```

```
// — POLYMORPHIC REPORT COMPILATION —
System.out.println("\n=====");
System.out.println("          GENERATING REPORT CARD          ");
System.out.println("=====");
System.out.println("Account File:   " + currentStudent.getName());
System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

// If the object is a GraduateStudent, show their thesis topic using a safe type cast check
if (currentStudent instanceof GraduateStudent) {
    GraduateStudent castedGrad = (GraduateStudent) currentStudent;
    System.out.println("Research Focus:  " + castedGrad.getThesisTitle());
}

double averageGrade = currentStudent.calculateAverage();
System.out.println("Final Grade:    " + averageGrade + "%");

// Runtime Magic: Automatically triggers the correct 90% or 95% target rules
if (currentStudent.qualifiesForHonorRoll()) {
    System.out.println("Honor Roll:      ★ APPROVED ★");
} else {
    System.out.println("Honor Roll:      Standard Academic Standing");
}
System.out.println("=====");
}

inputScanner.close();
}
}
```

Task 9: File Input Output

Instead of just printing the final report card text to your screen using `System.out`, we will simultaneously write that exact text data to a permanent file named **report_cards.txt**. Each time you add a new student, the program will **append** (add onto) the bottom of that file so your history is never lost.

Note:

No changes to class `Student` and class `GraduateStudent` for adding File Input Output. Only adjust main method to automatically save these report cards to your hard drive, we will introduce **File I/O** (Input/Output) using Java's built-in `FileWriter` and `IOException` error handling.

What We Are Adding in This Final Step:

1. **New Package Imports:** We add `import java.io.FileWriter;` and `import java.io.IOException;` to the very top of our file to pull Java's file-writing tools off the digital library shelf.
2. **The `FileWriter` Tool:** We open a writer channel inside our report generation segment. By passing `true` as a configuration parameter (`new FileWriter("report_cards.txt", true)`), we tell Java to update and append to the text file rather than wiping it clean every loop cycle.
3. **Mandatory File Try-Catch:** Writing files to a hard drive carries real-world risks (like your hard drive running out of space or being locked). Java forces us to wrap our file-writing code inside a try-catch block to handle an `IOException` safely without crashing.

Verifying Your Work

Once you compile and run this program, add a couple of student records, and select option 3 to exit.

Look inside your project's main folder—you will see a brand-new file named `report_cards.txt` created entirely by your code. If you open it with Notepad or any basic text editor, you will find your clean academic records saved exactly as specified on your disk [local].

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println("  WELCOME TO THE GLOBAL STUDENT ENGINE  ");
        System.out.println("=====");

        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear scanner buffer

            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break;
            }

            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("✗ Invalid option. Please select 1, 2, or 3.");
                continue;
            }

            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            Student currentStudent = null;

            if (menuChoice == 1) {
                currentStudent = new Student(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                currentStudent = new GraduateStudent(rawName, thesis, inputScanner);
            }
        }
    }
}
```

```
// --- POLYMORPHIC REPORT COMPILATION ---
    System.out.println("\n=====");
    System.out.println("          GENERATING REPORT CARD          ");
    System.out.println("=====");
    System.out.println("Account File:      " + currentStudent.getName());
    System.out.println("Tracked Classes:  " + currentStudent.getGrades().length);

    String extraDetails = "";
    if (currentStudent instanceof GraduateStudent) {
        GraduateStudent castedGrad = (GraduateStudent) currentStudent;
        extraDetails = "Research Focus:  " + castedGrad.getThesisTitle() + "\n";
        System.out.print(extraDetails);
    }

    double averageGrade = currentStudent.calculateAverage();
    System.out.println("Final Grade:      " + averageGrade + "%");

    String honorStatus = "Standard Academic Standing";
    if (currentStudent.qualifiesForHonorRoll()) {
        honorStatus = "★ APPROVED FOR HONOR ROLL ★";
    }
    System.out.println("Honor Roll:      " + honorStatus);
    System.out.println("=====");

    // --- NEW STEP: AUTOMATICALLY SAVE DATA TO HARD DRIVE VIA FILE I/O ---
    // Setting the second parameter to 'true' enables APPEND mode so old entries stay saved!
    try (FileWriter fileWriter = new FileWriter("report_cards.txt", true)) {

        // Write data patterns onto the disk exactly how we formatted them on screen
        fileWriter.write("=== STUDENT RECORD ===\n");
        fileWriter.write("Name: " + currentStudent.getName() + "\n");
        fileWriter.write("Average: " + averageGrade + "%\n");
        if (!extraDetails.isEmpty()) {
            fileWriter.write(extraDetails);
        }
        fileWriter.write("Status: " + honorStatus + "\n");
        fileWriter.write("-----\n\n");

        System.out.println("💾 Backup Status: Report successfully written to 'report_cards.txt'.");

    } catch (IOException e) {
        // Intercept data path anomalies (like read-only file lockouts or hardware drops)
        System.out.println("❌ Local Storage Failure: Could not write record backup to disk drive.");
    }

    inputScanner.close();
}
}
```

Task 10: Read back from on start-up.

- Read your saved records back into the program when the application boots up, use Java's built-in File and Scanner tools.
- Instead of jumping straight into the interactive loop, the program will look for report_cards.txt the exact moment it starts. If the file exists, the program should open it, read it line-by-line, and print the historical records directly to the screen before presenting the registration menu.

What We Are Adding in This Step:

1. **New Package Import:** We add import java.io.File; to manage file path lookups on your operating system.
2. **Boot-Up Initializer Function:** We introduce a new public static method called **loadPastRecords()** outside of main.
3. **The File Reading Process:** We pass a File object into a new Scanner. The scanner uses a while (fileScanner.hasNextLine()) loop to read every single line of text from the file until it hits the very end.
4. **Zero Code Discarded:** The FileWriter append logic, object structure, and error-handling mechanics remain completely active.

How to Test this Evolution:

1. Compile and run the code. If it's your very first time running it with this updated logic, it will say "No historical database found"
2. Add a student (e.g., "Sam Wilson") and let the program log their grades to disk.
3. Exit the application using choice 3.
4. Run the program a **second time**. Notice what happens instantly on boot-up! The loadPastRecords() block finds the file and outputs Sam's entire profile right back onto your console interface before asking you for new inputs.

Future improvements

- Learn about ArrayLists: Upgrade UnifiedTracker.java so that instead of just flashing reports on screen, the system keeps an active in-memory list tracking every student registered during that loop cycle.
- Log error for duplicate entry.
- Add student member variables for student ID, Second name etc.
- Add UI for user interaction.

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {

    // — NEW STEP: FUNCTION TO LOAD PAST RECORDS FROM LOCAL DISK —
    public static void loadPastRecords() {
        File databaseFile = new File("report_cards.txt");

        // 1. Check if the backup file actually exists on the computer
        if (!databaseFile.exists()) {
            System.out.println("❗ No historical database found. Starting with a blank registry.\n");
            return; // Exit function early if there is nothing to read
        }

        System.out.println("📁 Loading historical archives from disk drive...");
        System.out.println("=====");

        // 2. Open a specialized scanner locked onto the physical file instead of System.in
        try (Scanner fileScanner = new Scanner(databaseFile)) {

            // 3. Keep loop active as long as there are text strings left to read in the file
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line); // Output historical file lines directly to console
            }
            System.out.println("=====");
            System.out.println("✅ Data successfully loaded into memory view!\n");

        } catch (IOException e) {
            // Guard against system errors during active read runtime
            System.out.println("❌ Local Storage Failure: Could not safely parse text files.");
        }
    }

    public static void main(String[] args) {
        // — BOOT-UP REUSE: FETCH OLD HISTORY THE MOMENT APPLICATION STARTS —
        loadPastRecords();
    }
}
```